

CryptoSim

Симулатор за търговия с криптовалути

Daniel Stefanov · ФН OMI0600397

Fullstack Application Development with Node.js + Express.js + React.js

 <https://github.com/danielstefanov212/CryptoSim>

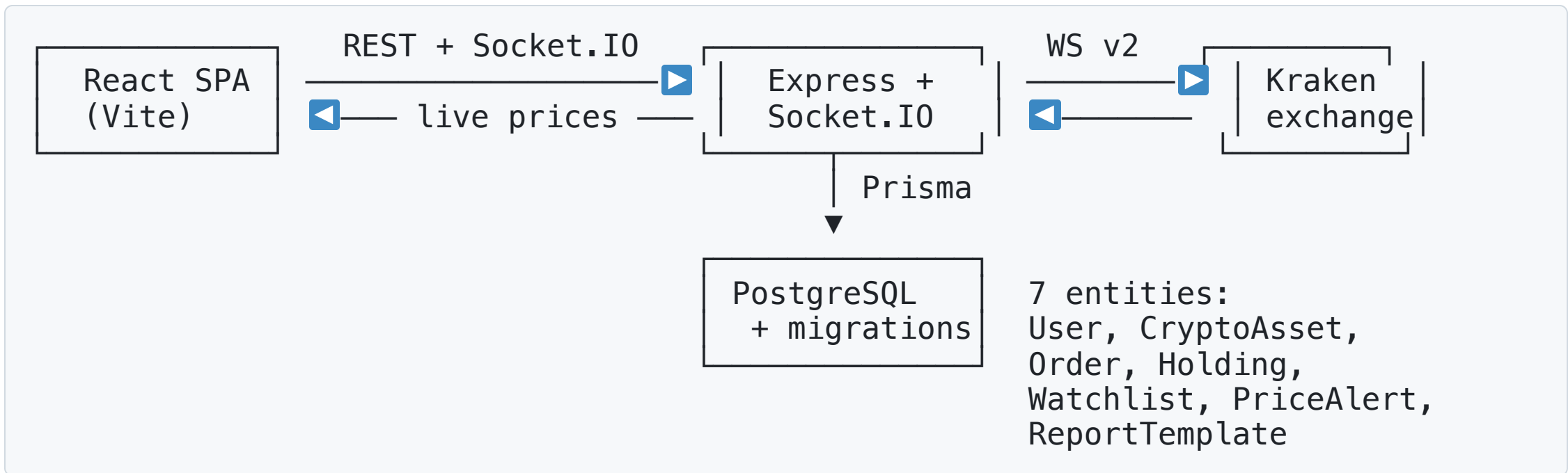
Какво представлява проектът?

- **Безрискова среда** за упражняване на търговия с крипто
- Всеки потребител започва с виртуални **\$10 000**
- Реални цени в реално време от борсата **Kraken**
- Покупка/продажба, портфолио, watchlist, ценови **alerts**
- **Запазваеми отчети** на стойността на портфейла във времето с **PDF експорт**
- Админ панел за управление на потребители и активи

Технологичен стек

Слой	Технологии
Frontend	React + Vite + TypeScript, Chart.js, React Router, Socket.IO client
Backend	Node.js 20, Express, Socket.IO, TypeScript
База данни	PostgreSQL 16 + Prisma ORM
Реално време	Kraken WebSocket v2 (тикове) + REST OHLC (история за отчети)
Сигурност	JWT + bcrypt, Zod валидация, CORS allow-list
Decimal математика	<code>Prisma.Decimal</code> / <code>decimal.js</code> (без floating-point грешки)
PDF	<code>pdfkit</code> + <code>chartjs-node-canvas</code> (server-side chart)

Архитектура



- Public Socket.IO namespace → топ-3 цени за анонимни посетители
- Private Socket.IO namespace → JWT handshake, per-user стая за alerts

Данни и атомарност

7 entity-та: `User`, `CryptoAsset`, `Order`, `Holding`, `Watchlist`, `PriceAlert`, `ReportTemplate`.

Сделките са атомарни — `prisma.$transaction`:

- BUY: cash↓, holding↑, нов Order — всичко или нищо
- SELL: `SELECT ... FOR UPDATE` на user-а първо → предотвратява dead-lock
- Race условия → 409 `CONCURRENT_TRADE`

Парите са `Decimal` — никакви float грешки в сметките.

Alerts (известия в реално време)

1. Потребител създава `PriceAlert` (символ, target price, ABOVE/BELOW)
2. **AlertEngine** (in-memory индекс) → автоматично се абонира за този символ от Kraken
3. На всеки tick → $O(N)$ сравнение по символ с `Decimal.gte / lte`
4. При trigger → **атомичен** `updateMany({ isTriggered: false })` (защита от двойно firing)
5. Engine emit-ва вътрешен event → Socket.IO го изпраща в user-овата стая
6. Frontend хук показва **native desktop notification** (Web Notifications API)

Refcounted Kraken абонаменти — много alerts на BTC → една абонамент upstream.

Отчети (Portfolio-Value-Over-Time)

Запазваем **template** с два режима:

- Fixed start date — закотвена дата
- Rolling — "последните N дни" (плъзга се при всеки run)

Runner алгоритъм:

1. Резолване на window (clamp на бъдещи дати)
2. Избор на granularity: hourly $\leq$ 24ч, иначе daily/weekly/monthly (до 720 точки)
3. **Promise.allSettled** на Kraken OHLC fetch — провал на един символ $\rightarrow$ data gap, не цял отчет
4. Replay на order ledger-а през оста на времето с decimal математика
5. Честни **null** стойности при липсваща история $\rightarrow$ chart показва прекъсване, не \$0

PDF експорт — същата Chart.js конфигурация рендерирана server-side.

Sigurnost и валидация (на четири слоя)

Слой	Какво
CORS	allow-list <code>localhost:5173,5174</code>
JWT auth	<code>jsonwebtoken</code> , <code>bcrypt hash</code> на пароли, <code>expiry guard</code>
Zod schemas	Валидация на всеки request payload в Express middleware
Service guards	<code>assertKnownSymbol</code> , <code>assertWindowOrdering</code> , <code>assertDatesNotInFuture</code>
Owner-scoped 404	Cross-user IDs не са изброими (връща 404, не 403)
Атомарни ъпдейти	<code>updateMany({where: ..., isTriggered: false})</code> срещу double-fire

Какво научих и какво следва

Какво работи добре:

- Decimal математика → нулеви парични грешки
- In-memory alert engine → ниска латентност
- Refcounted subscriptions → ефективно използване на Kraken WS
- Server-side PDF rendering → консистентен export

Възможни следващи стъпки:

- Кеширане на исторически OHLC (по-малко зависимост от Kraken)
- Лимитни поръчки (limit / stop-loss), не само пазарни
- Сравнение с benchmark (vs hold-only-BTC)

Благодаря за вниманието!

 **GitHub:** <https://github.com/danielstefanov212/CryptoSim>

Daniel Stefanov · ФН 0MI0600397